

一、实验名称：二叉树遍历实验

二、实验目的

掌握二叉树的“先根”遍历存储表示原理。

掌握二叉树“先根”遍历存储表示转换为二叉链表表示的原理和方法。

掌握采用二叉链表表示的二叉树的先根遍历、中根遍历、后根遍历实现方法。

三、实验要求

熟悉 C++ 语言编程。

掌握二叉树遍历原理。

四、实验内容

（一）问题描述

给定一棵按“先根”遍历存储表示的二叉树，对这棵二叉树进行先根遍历、中根遍历、后根遍历。

（二）算法

按“先根”遍历存储表示的二叉树中，在遍历过程中，若发现子树为空，输出 0。例如：A 有左子树 B，没有右子树，其“先根”遍历存储表示为 A B 0 0 0。

又如：A 为根，B 为 A 的左孩子，D 为 A 的右孩子，C 为 B 的左孩子，其“先根”遍历存储表示为 A B C 0 0 0 D 0 E 0 0。

“先根”遍历存储表示转换为二叉链表表示的规则：在“先根”遍历存储表示中，根结点后面结点如果为空，则没有左子树；如果再后面结点有效，则为根的右孩子；否则根也是叶子（即后跟 2 个空结点）。根结点后，如果结点为有效结点，则该结点为根结点的左孩子。

（三）输入

第一行：先根遍历的二叉树结点数目（包括空结点）。

第二行：n 个“先根”遍历存储表示的二叉树字符序列（用空格隔开）（字符 0 表示空结点）。

（四）输出

第 1 行：二叉树的先根遍历序列。

第 2 行：二叉树的中根遍历序列。

第 3 行：二叉树的后根遍历序列。

（五）输出样本

ABCDE

CBADE

CBEDA

五、实验原理

二叉树的遍历是指按照某种顺序访问二叉树中的每个结点，使得每个结点被访问一次且仅被访问一次。本次实验涉及三种主要的遍历方式：

先根遍历（Pre-order Traversal）：先访问根结点，然后遍历左子树，最后遍历右子树。

中根遍历（In-order Traversal）：先遍历左子树，然后访问根结点，最后遍历右子树。

后根遍历（Post-order Traversal）：先遍历左子树，然后遍历右子树，最后访问根结点。

实验中，首先需要将按“先根”遍历存储表示的二叉树转换为二叉链表表示。二叉链表中每个结点包含数据域和两个指针域，分别指向左孩子和右孩子。通过递归的方式，根据“先根”遍历序列创建二叉链表，对于序列中的“0”，表示对应位置为空结点，不创建结点且指针指向空。

六、实验步骤

1 二叉链表定义：定义二叉树结点的结构体，包含数据域和左、右孩子指针。

```
struct BiNode {  
    char data;  
    BiNode *lchild, *rchild;};  
BiNode *BiTree;
```

2 “先根” 遍历表示到二叉链表表示的转换：使用递归函数 `CreateBiTree` 实现转换，通过全局变量 `NodeID` 跟踪当前处理的序列位置。当遇到 “0” 或序列结束时，返回空指针；否则创建新结点，递归创建左、右子树。

`int NodeID`; // 全局变量，表示 “先根” 遍历存储表示序列序号

```
BiNode *CreateBiTree(char *c, int n) {  
    BiNode *T;  
    NodeID++;  
    if (NodeID > n) return NULL;  
    if (c[NodeID] == '0') return NULL;  
    // 生成一个结点  
    T = new BiNode;  
    T->data = c[NodeID];  
    T->lchild = CreateBiTree(c, n);  
    T->rchild = CreateBiTree(c, n);  
    return T;}
```

3 先根遍历函数：递归实现先根遍历，先访问当前结点数据，再递归遍历左子树，最后递归遍历右子树。

```
void PreOrderTraverse(BiNode *T) {  
    if (T == NULL) return;  
    cout << T->data;  
    PreOrderTraverse(T->lchild);  
    PreOrderTraverse(T->rchild);}
```

4 中根遍历函数：递归实现中根遍历，先递归遍历左子树，再访问当前结点数据，最后递归遍历右子树。

```
void InOrderTraverse(BiNode *T) {  
    if (T == NULL) return;  
    InOrderTraverse(T->lchild);  
    cout << T->data;  
    InOrderTraverse(T->rchild);}
```

5 后根遍历函数：递归实现后根遍历，先递归遍历左子树，再递归遍历右子树，最后访问当前结点数据。

```
void PostOrderTraverse(BiNode *T) {  
    if (T == NULL) return;  
    PostOrderTraverse(T->lchild);
```

```
PostOrderTraverse(T->rchild);  
cout << T->data;}
```

6 主程序：读取输入的结点数和序列，调用创建二叉树函数，然后分别调用三种遍历函数并输出结果。

```
int main() {  
    int i, SampleNum;  
    char c[100];  
    cin >> SampleNum; // 输入二叉树结点样本数目  
    for (i = 1; i <= SampleNum; i++) cin >> c[i]; // 输入二叉树结点数据  
    NodeID = 0;  
    BiTree = CreateBiTree(c, SampleNum); // 创建二叉树  
    PreOrderTraverse(BiTree); cout << endl; // 先根遍历二叉树  
    InOrderTraverse(BiTree); cout << endl; // 中根遍历二叉树  
    PostOrderTraverse(BiTree); cout << endl; // 后根遍历二叉树  
    return 0;}
```

七、实验代码

```

1  #include <iostream>
2  using namespace std;
3
4  struct BiNode {
5      char data;
6      BiNode *lchild, *rchild;
7  };
8  BiNode *BiTree;
9  int NodeID; // 全局变量，表示“先根”遍历存储表示序列序号
10
11 BiNode *CreateBiTree(char *c, int n) {
12     BiNode *T;
13     NodeID++;
14     if (NodeID > n) return NULL;
15     if (c[NodeID] == '0') return NULL;
16     // 生成一个结点
17     T = new BiNode;
18     T->data = c[NodeID];
19     T->lchild = CreateBiTree(c, n);
20     T->rchild = CreateBiTree(c, n);
21     return T;
22 }
23
24 void PreOrderTraverse(BiNode *T) {
25     if (T == NULL) return;
26     cout << T->data;
27     PreOrderTraverse(T->lchild);
28     PreOrderTraverse(T->rchild);
29 }
30
31 void InOrderTraverse(BiNode *T) {
32     if (T == NULL) return;
33     InOrderTraverse(T->lchild);
34     cout << T->data;
35     InOrderTraverse(T->rchild);
36 }
37
38 void PostOrderTraverse(BiNode *T) {
39     if (T == NULL) return;
40     PostOrderTraverse(T->lchild);
41     PostOrderTraverse(T->rchild);
42     cout << T->data;
43 }
44
45 int main() {
46     int i, SampleNum;
47     char c[100];
48     cin >> SampleNum; // 输入二叉树结点样本数目
49     for (i = 1; i <= SampleNum; i++) cin >> c[i]; // 输入二叉树结点数据
50     NodeID = 0;
51     BiTree = CreateBiTree(c, SampleNum); // 创建二叉树
52     PreOrderTraverse(BiTree); cout << endl; // 先根遍历二叉树
53     InOrderTraverse(BiTree); cout << endl; // 中根遍历二叉树
54     PostOrderTraverse(BiTree); cout << endl; // 后根遍历二叉树

```

八、实验结果与分析

（一）实验结果

以输入样本“11ABC000D0E00”为例，实验输出如下：

先根遍历序列：ABCDE

中根遍历序列：CBADE

后根遍历序列：CBEDA

（二）结果分析

实验结果与预期的输出样本一致，表明实验代码能够正确实现二叉树的创建和三种遍历方式。在转换过程中，递归函数 CreateBiTree 正确处理了“0”所表示的空结点，准确构建了二叉链表。三种遍历函数按照各自的遍历顺序递归访问结点，输出了正确的遍历序列。

通过实验可以看出，二叉树的遍历顺序直接影响访问结点的先后次序，先根遍历先访问根结点，中根遍历在左子树遍历完成后访问根结点，后根遍历则在左右子树都遍历完成后访问根结点。这种特性使得不同的遍历方式适用于不同的应用场景，例如中根遍历常用于二叉搜索树中按升序或降序获取元素。

九、实验总结与体会

本次实验加深了对二叉树遍历原理和实现方法的理解。通过将“先根”遍历存储表示转换为二叉链表表示，掌握了二叉树的存储转换方法，体会到递归在二叉树操作中的重要作用。在实现先根、中根、后根遍历函数的过程中，进一步理解了三种遍历方式的逻辑差异和递归实现的关键。

实验过程中，需要注意递归函数的边界条件处理，例如当序列结束或遇到“0”时应返回空指针，避免出现无限递归或访问空指针的错误。同时，全局变量 `NodeID` 的使用需要谨慎，确保在创建二叉树时能够正确跟踪序列的位置。

通过本次实验，不仅提高了 C++ 编程能力，尤其是对指针和递归的运用，还培养了分析问题和解决问题的能力。在今后的学习中，将进一步深入学习二叉树的其他操作和应用，巩固数据结构的知识。

完成上述基本实验要求的同学**鼓励**完成非递归遍历算法的实现，可单独一个 word 文件模仿上面格式完成（选作内容）